AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF COMPUTING, BENGALURU
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
IV SEMESTER AIE (Jan - May 2026)

DESIGNANDANALYSIS OF ALGORITHMS COURSE

PLAN (L-T-P-C: 2-0-3-3)

Lab Questions:
Sorting and DAC
[Note: Analyze the complexity]

1. You are working as a Software Engineer at **QuickCart**, a fast-growing e-commerce company. The platform has multiple warehouses across different cities. Each warehouse maintains a **sorted list of order dispatch times (in minutes)** for the day.

   At the end of the day, the company wants to generate a **single master dispatch timeline** that shows all orders from all warehouses in **globally sorted order** so they can analyze delivery efficiency. However, since the number of warehouses and orders is large, the solution must be **efficient and scalable**.

   INPUT: You are given **m sorted arrays**, each containing **n dispatch times**:
   Warehouse 1: [10, 20, 30, 40]
   Warehouse 2: [15, 25, 35, 45]
   Warehouse 3: [27, 29, 37, 48]
   Warehouse 4: [31, 34, 40, 51]
   Warehouse 5: [18, 19, 22, 28]
   OUTPUT: [10, 15, 18, 19, 20, 22, 25, 27, 28, 29, 30, 31, 34, 35, 37, 40, 40, 45, 48, 51]

2. You are working as a Software Engineer at PayPal. The company processes millions of transactions daily. At the end of each day, the analytics team needs to identify the **top K highest transaction amounts** to detect **premium customers and potential fraud cases**.

   Since the number of transactions (**N**) is extremely large (millions), and **K is very small compared to N**, the solution must be highly efficient in terms of **time and memory**.

   INPUT: An array representing transaction amounts:
   [1, 23, 11, 8, 31, 4, 50, 60, 2, 70] with K=3
   OUTPUT – 70, 60, 50

3. You are working as a Software Engineer at Netflix. The platform tracks whether users are **inactive (0)** or **active (1)** during a specific time window. For performance optimization, the analytics team wants all **inactive users (0)** to appear first in the database, followed by **active users (1)**, so that different processing strategies can be applied efficiently. However, the dataset is very large, and the segregation must be done **in-place** with **minimum time and space complexity**.

   INPUT: USER Activity = [0, 1, 1 ,0 ,0, 1, 1, 1, 0, 1]

4. You are working as a Software Engineer at Microsoft. The company has only **one high-priority conference room** in a building, and multiple teams have requested to use it for different meetings.

   Each meeting has a **start time** and **finish time**. Since only one meeting can be held in the room at a time, your task is to help the admin **schedule the maximum number of non-overlapping meetings** in that room.

   Your goal is to design an efficient algorithm to select the **maximum number of meetings that can be conducted in a single day**.

   INPUT: List of meeting time intervals: (1,4), (3,5) (0,6), (5,), (3,8), (5,9), (6,10), (8, 11), (8,12), (2,13), (12,14)
   OUTPUT: (1,4), (5,), (8, 11), (12, 14)

5. You are working as a Software Engineer at Google in the Search Quality team. Your team is responsible for evaluating how much the **new ranking algorithm differs from the old ranking algorithm**.

   Each website is assigned a **ranking score**, and ideally, higher-quality websites should appear before lower-quality ones. However, sometimes due to algorithm issues, **lower-ranked websites appear before higher-ranked ones**, causing ranking errors. These errors are called **Ranking Inversions**.

   Your task is to **count how many such inversions exist**, which helps measure how inaccurate the ranking is.

   INPUT: An array representing ranking scores (non-negative integers):
   A ={10 ,1,2,4,13,9,5}
   The number of inversions that are possible as follows:
   {(10, 1), (10, 2),(10 , 4), (10, 9), (10, 5) , (13 , 9), (13, 5) , (9 , 5) }
   Total count of inversions are: 8

6. You are working as a Software Engineer at Amazon in the Pricing Analytics team. Your system receives an unsorted array representing the **profit or loss values (in dollars)** of different products sold during a flash sale. Positive numbers represent **profit**. Negative numbers represent **loss**

   Your manager wants to identify the **pair of products whose combined product value is maximum**, because this combination indicates the **strongest impact on overall revenue (positive × positive or negative × negative)**. You need to design an **efficient algorithm** to find this maximum product pair.

   Input: Arr=[1,7,4,2,8,6,3,9, 5]
   Output = 72

7. You are working as a Software Engineer at Tesla on the Autonomous Driving team. Each self-driving car continuously records **temperature readings (in °C)** from its battery sensors during a trip. For safety analysis, the system must quickly determine:

The **minimum temperature** (to detect freezing risk)

The **maximum temperature** (to detect overheating risk)

Since the car generates **millions of readings**, your algorithm must be **highly efficient** and use **minimum comparisons**.

Input: An array of sensor readings: arr = [34, 12, 45, 67, 23, 9, 56, 89, 15]

OUPUT: 9, 89

8. You are working as a Software Engineer at Meta. The platform generates **millions of user activity logs per second**, and these logs must be sorted quickly for real-time analytics.

   Initially, your team used **Quick Sort** because of its fast average performance. However, in some cases (when data is nearly sorted or badly distributed), Quick Sort performance degraded to **$O(N^2)$**, causing system slowdowns.

   To solve this, your team decided to implement **Introsort**, a hybrid sorting algorithm that:

   - Starts with **Quick Sort**
   - Switches to **Heap Sort** when recursion depth becomes too large
   - Uses **Insertion Sort** for small subarrays

   This ensures **guaranteed O(N log N) worst-case performance** while maintaining fast average speed. You are asked to implement this algorithm. An unsorted array of log timestamps: arr = [24, 12, 18, 5, 42, 30, 7, 15, 60, 1]

   OUTPUT: [1, 5, 7, 12, 15, 18, 24, 30, 42, 60]

9. You are working as a Software Engineer at Emirates. The airline uses an automated system to generate a **boarding priority list** for passengers based on factors like loyalty status, ticket class, and special assistance. Ideally, the list should be **fully sorted by priority**. However, due to last-minute upgrades, seat changes, and manual overrides, the final list becomes **k-nearly sorted**, meaning:

   Each passenger is at most **k positions away** from their correct sorted position, where **k is much smaller than N**.

   Your manager wants you to efficiently sort this list before boarding begins, ensuring minimal delay.

   Input

   An example boarding priority array:
   A = [101, 103, 102, 104, 106, 105, 107]
   Here:

N = 7 passengers

k = 1 or 2 (each element is only slightly misplaced)

a) You are asked to evaluate the following sorting algorithms:

- Bubble Sort
- Selection Sort
- Insertion Sort

Identify which sorting technique is best suited for sorting a k-nearly sorted array when k << N**?** Justify your answer using **detailed time complexity analysis** for: Best case, worst case and average case.

Also explain why some algorithms do unnecessary comparisons? Why one algorithm performs better when the array is nearly sorted. The **best chosen algorithm**

b. Implement the traditional and the improved versions of insertion sort and plot graphs for complexity analysis and comparisons.

c. Discuss the impact of the improved insertion sort on the best case and average case with complexity analysis. It is helpful in such cases.? You can choose to plot graphs and check.